Creación y configuración de las máquinas virtuales, paso a paso

AGR — Práctica 2

Pablo Elvira Calatayud

Índice

- Parte 1: VM del servidor web con Vagrant
  - Paso 1. Elección de la imagen y del proveedor
  - Paso 2. Parámetros
  - Paso 3. Red de la VM
  - Paso 4. Hardware virtual
  - Paso 5. Aprovisionamiento (scripts Bash dentro de la VM)
  - Paso 6. Qué ocurre al ejecutar `vagrant up`
- Parte 2: red del laboratorio 3103 con QEMU-KVM
  - Paso 0. Definir la topología
  - Paso 1. Obtener la imagen de partida
  - Paso 2. Construir la imagen base común (una sola vez)
  - Paso 3. Crear las redes en el anfitrión
  - Paso 4. Disco de cada máquina
  - Paso 5. Configurar cada disco antes del primer arranque
  - Paso 6. Generar el XML de cada dominio
  - Paso 7. Definir y arrancar
  - Paso 8. Qué ocurre dentro de cada VM al arrancar
  - Paso 9. Espera y verificación

Este documento explica cómo se crean y configuran todas las máquinas virtuales de la práctica: qué hace cada herramienta, en qué orden y qué ejecuta internamente. La parte 1 es una VM en VirtualBox creada con Vagrant (`vagrant up`); la parte 2, nueve VMs en QEMU-KVM creadas con `parte2-kvm/agr_kvm.py crear`. Los fragmentos de código de este documento muestran lo que hacen las herramientas; no hay que ejecutarlos a mano.

Parte 1: VM del servidor web con Vagrant

Paso 1. Elección de la imagen y del proveedor

- **Proveedor:** VirtualBox. Se fuerza en el `Vagrantfile` (`ENV["VAGRANT_DEFAULT_PROVIDER"] ||= "virtualbox"`), de modo que si VirtualBox no funciona Vagrant da el motivo concreto en lugar de un error genérico.
- **Box:** `bento/ubuntu-22.04`, una imagen Ubuntu Server mínima mantenida por el proyecto Bento, con las *Guest Additions* y el usuario `vagrant` preparados.

Paso 2. Parámetros

Al principio del `Vagrantfile` se definen como constantes todos los valores de la práctica, para cambiarlos en un solo sitio:

```
VM_NAME      = "agr-nodeserver"
VM_BOX       = "bento/ubuntu-22.04"
VM_MEMORY    = (ENV["VM_MEMORY"] || 1024).to_i
VM_CPUS      = (ENV["VM_CPUS"] || 1).to_i
IP_SERVICIO  = "192.168.56.10"   # red privada host-only (servicio web)
IP_GESTION   = "192.168.57.10"   # segunda red privada host-only (gestión)
PUERTO_GUEST = 3000              # puerto del servidor dentro de la VM
PUERTO_HOST  = 8080              # puerto expuesto en el anfitrión
REPO_URL     = "https://github.com/gisai/SSR-master-server.git"
```

Paso 3. Red de la VM

```
config.vm.network "forwarded_port", guest: 3000, host: 8080, protocol: "tcp"
config.vm.network "private_network", ip: "192.168.56.10"
config.vm.network "private_network", ip: "192.168.57.10"
config.vm.network "public_network", bridge: BRIDGE_IF if BRIDGE_IF
```

NIC	Tipo VirtualBox	Uso
1	NAT (la crea Vagrant siempre)	Salida a Internet, SSH de Vagrant y **port forwarding 8080 → 3000**
2	host-only `192.168.56.10/24`	Red privada del servicio
3	host-only `192.168.57.10/24`	Red privada de gestión (VM con varias interfaces)
4 (opcional)	bridge	Red pública por DHCP, sólo si se define `BRIDGE_IF`

Sin `host_ip`, VirtualBox escucha el puerto 8080 en todas las interfaces del anfitrión, así que el servicio es accesible también desde otros equipos.

Paso 4. Hardware virtual

```
        config.vm.provider "virtualbox" do |vb|
          vb.name   = "agr-nodeserver"
          vb.memory = 1024
          vb.cpus   = 1
          vb.linked_clone = true                                  # disco diferencial
          vb.customize ["modifyvm", :id, "--cpuexecutioncap", "80"]
          vb.customize ["modifyvm", :id, "--natdnshostresolver1", "on"]
        end
```

`linked_clone` hace que la VM no copie el disco de la *box*, sino que cree un disco diferencial sobre ella (`Snapshots/{uuid}.vmdk`).

Paso 5. Aprovisionamiento (scripts Bash dentro de la VM)

Vagrant ejecuta como root, la primera vez que se hace `vagrant up` (o con `vagrant provision`), cuatro scripts en orden:

1. **`sistema`**
   - Escribe `/etc/default/keyboard` con `XKBLAYOUT="es"` y aplica el teclado español con `dpkg-reconfigure keyboard-configuration`, `setupcon`, `localectl` y `loadkeys es`.
   - Genera el locale `es_ES.UTF-8` y fija la zona horaria `Europe/Madrid`.
   - Instala `git`, `curl`, `ufw`, `htop`, `sysstat` y `net-tools`.
   - Configura `ufw`: deniega la entrada por defecto y abre sólo 22/tcp y 3000/tcp.
2. **`node`**: añade el repositorio de NodeSource e instala Node.js 22.
3. **`servidor`**
   - Crea el usuario de sistema `nodeapp` y clona (o actualiza con `git pull`) el repositorio en `/opt/ssr-master-server`.
   - Crea `public/logs` y ejecuta `npm install --omit=dev`.
   - Crea y arranca la unidad systemd `ssr-server.service`, que ejecuta `node app.js` como `nodeapp` y se reinicia si falla.
   - Espera a que `http://localhost:3000` responda.
4. **`login-info`**
   - Desactiva el MOTD de Ubuntu.

- Instala `/etc/profile.d/99-servidor-info.sh`, que en cada inicio de sesión interactivo imprime sistema, red, puertos, estado del servicio, URLs y reglas del cortafuegos.

Por qué `node app.js` y no `npm start`. `npm start` ejecuta `bin/www`, que llama a `server.listen(3000)`, pero `app.js` ya había hecho `app.listen(3000)`. El segundo `listen` falla con `EADDRINUSE` y el proceso termina.

Paso 6. Qué ocurre al ejecutar `vagrant up`

1. Descarga la *box* (la primera vez) a `~/.vagrant.d/boxes`.
2. La importa en VirtualBox como VM maestra y crea un clon enlazado `agr-nodeserver`.
3. Configura las NICs, la redirección de puertos, la RAM y la CPU.
4. Arranca la VM sin interfaz gráfica, espera a SSH y sustituye la clave insegura por una propia.
5. Configura las IPs de las redes privadas y monta `/vagrant`.
6. Ejecuta los cuatro *provisioners* y muestra el `post_up_message` con las URLs.

Parte 2: red del laboratorio 3103 con QEMU-KVM

El proceso completo lo ejecuta `sudo ./agr_kvm.py crear`. A continuación se detalla cada paso y lo que el script ejecuta en él.

Paso 0. Definir la topología

En `topologia.json` está el plan de numeración del laboratorio (`doc/development/enunciado/plan_numeracion_3103.pdf`):

- **9 redes**, cada una con su bridge y su CIDR.
- **9 máquinas**, cada una con su rol (`router`, `server` o `pc`), RAM, vCPUs, interfaces (nombre, red e IP) y rutas estáticas.

Decisiones de diseño:

- Las cuatro redes inferiores de 8 PC se representan con **un PC cada una**, como permite el enunciado.
- **El anfitrión es el "PC Profesor"** (10.200.0.2), conectado a RA por la red 10.200.0.0/30 del plano.
- Las interfaces de los routers **se llaman como en el plano**. Linux no admite `/` en los nombres de interfaz, así que `ge0/0/1` pasa a ser `ge0-0-1`.
- Cada interfaz tiene una **MAC determinista** `52:54:00:aa:<nº máquina>:<nº interfaz>`, que permite identificarla dentro de la VM sin depender del orden PCI.

Antes de desplegar, el script valida que cada IP pertenece a la red a la que se conecta.

Paso 1. Obtener la imagen de partida

Como recomienda el enunciado, se parte de la **imagen de disco de Moodle** (`agr-vm-base.qcow2`): Ubuntu 20.04.5 LTS, kernel 5.4, 4 GB virtuales, usuario `root` con contraseña `agr`, interfaz `eth0` gestionada por ifupdown y cloud-init. El script la busca en `parte2-kvm/imagen/agr-`

`vm-base.qcow2` y, si no está, la descarga del Release `imagen-moodle` del repositorio (no se guarda en git por su tamaño). **La imagen original nunca se modifica**: se trabaja sobre una copia.

Paso 2. Construir la imagen base común (una sola vez)

Todas las VMs comparten **una imagen base con todo el software instalado**. Los servicios propios de cada rol quedan deshabilitados y se activan después en cada VM.

```
qemu-img convert -O qcow2 imagen/agr-vm-base.qcow2 agr-base.qcow2   # copia
qemu-img resize agr-base.qcow2 6G
virt-customize -a agr-base.qcow2 --network \
  --run-command '<amplía la partición raíz con growpart y resize2fs>' \
  --root-password password:agr --timezone Europe/Madrid \
  --upload guest/agr-net.sh:/usr/local/sbin/agr-net.sh \
  --upload guest/agr-net.service:/etc/systemd/system/agr-net.service \
  --upload guest/ssr-server.service:/etc/systemd/system/ssr-server.service \
  --upload guest/agr-info.sh:/etc/profile.d/agr-info.sh \
  --upload guest/build-base.sh:/root/build-base.sh \
  --run-command 'sh /root/build-base.sh'
```

`virt-customize` (de libguestfs) monta el disco en un pequeño entorno aislado y ejecuta órdenes dentro de la imagen como si fuera su sistema, **sin arrancarla**.

`guest/build-base.sh` hace, dentro de la imagen (con los repositorios de Ubuntu 20.04: FRR 7.2, Node.js 22 porque la glibc es 2.31):

1. **Paquetes:**
   - `frr` (FRRouting, la *Software Routing Suite*, sucesora de Quagga).
   - `qemu-guest-agent`.
   - Herramientas de red y sistema: `iproute2`, `iputils-ping`, `traceroute`, `tcpdump`, `curl`, `git`, `xz-utils` y `console-setup`.
2. **FRR:**
   - Activa `staticd` (el demonio de rutas estáticas).
   - Amplía los plazos de `watchfrr` (`-t 60 -T 90`).
   - Añade la unidad `frr.service.d/agr.conf`, que reaplica la configuración (`vtysh -b`) al arrancar.
   - Deja `frr` deshabilitado: sólo se activa en los routers.
3. **Node.js:**
   - Descarga los binarios oficiales en `/opt/node`.
   - Usa la versión 22, o la 16 si la imagen tiene una glibc anterior a 2.28.
4. **Servidor web:**
   - Crea el usuario `nodeapp`, clona SSR-master-server en `/opt/ssr-master-server` y ejecuta `npm install --omit=dev`.
   - Deja `ssr-server.service` deshabilitado.
5. **Red:**
   - Desactiva cloud-init, netplan, systemd-networkd y NetworkManager.
   - Habilita `agr-net.service`. A partir de aquí la red sólo la configura este servicio y nada intenta hacer DHCP en las redes del laboratorio.
6. **Teclado español** en `/etc/default/keyboard`.

7. **Limpieza:** borra la caché de apt y vacía `/etc/machine-id`, para que cada VM genere el suyo al arrancar.

Paso 3. Crear las redes en el anfitrión

Cada red del plano es un **bridge Linux** (equivalente a un switch). El anfitrión sólo tiene IP en `agr-prof`, porque hace de PC Profesor. El script ejecuta:

```
        for br in agr-srv agr-prof agr-ab agr-bc agr-bd agr-red1 agr-red2 agr-red3 agr-
red4; do
            ip link add name $br type bridge
            ip link set $br up
        done
        ip addr add 10.200.0.2/30 dev agr-prof                    # PC Profesor
        for net in 10.100.0.0/24 10.1.0.0/30 10.2.0.0/30 10.3.0.0/30 \
                   10.0.1.0/24 10.0.2.0/24 10.0.3.0/24 10.0.4.0/24; do
            ip route replace $net via 10.200.0.1 dev agr-prof     # todo el lab vía RA
        done
        # Que ufw/Docker no filtren el tráfico puenteado entre VMs
        echo 0 > /proc/sys/net/bridge/bridge-nf-call-iptables
```

Paso 4. Disco de cada máquina

Cada VM tiene un **disco qcow2 diferencial** cuya base (*backing file*) es la imagen común. Sólo guarda lo que esa VM cambia (unas decenas de MB por VM frente a los ~2 GB de la base). El script ejecuta:

```
        qemu-img create -f qcow2 -F qcow2 -b agr-base.qcow2
/var/lib/libvirt/images/agr/rc.qcow2
```

Paso 5. Configurar cada disco antes del primer arranque

Con `virt-customize`, sin red y en pocos segundos, el script escribe en cada disco su configuración. Ejemplo para RC:

```
        virt-customize -a rc.qcow2 \
          --hostname rc \
          --mkdir /etc/agr \
          --upload rc-node.conf:/etc/agr/node.conf \
          --upload hosts:/etc/hosts \
          --upload rc-frr.conf:/etc/frr/frr.conf \
          --run-command 'chown frr:frr /etc/frr/frr.conf && systemctl enable frr.service'
\
          --ssh-inject root:file:/home/<usuario>/.ssh/id_ed25519.pub
```

`/etc/agr/node.conf` (lo lee `agr-net.sh` en cada arranque):

```
role router
iface 52:54:00:aa:03:00 ge0-0-5 10.2.0.2/30
iface 52:54:00:aa:03:01 ge0-0-1 10.0.1.1/24
iface 52:54:00:aa:03:02 ge0-0-2 10.0.2.1/24
route 0.0.0.0/0 10.2.0.1
```

`/etc/frr/frr.conf` (sólo en los routers):

```
frr defaults traditional
hostname RC
log syslog informational
service integrated-vtysh-config
ip forwarding
!
interface ge0-0-5
 description Enlace RB - RC 10.2.0.0/30
exit
interface ge0-0-1
 description RED 1 (SW1) 10.0.1.0/24
exit
interface ge0-0-2
 description RED 2 (SW2) 10.0.2.0/24
exit
!
ip route 0.0.0.0/0 10.2.0.1
```

Según el rol:

Rol	Qué se activa
router	`frr.conf` con sus rutas estáticas + `systemctl enable frr`
server	`systemctl enable ssr-server`
pc	Nada más (la puerta de enlace la pone `agr-net.sh`)

En todas se escribe un `/etc/hosts` con los nombres de todos los nodos (`ping srv`, `ping rb-ge0-0-4`…) y la clave SSH pública del usuario.

Paso 6. Generar el XML de cada dominio

El XML de cada VM se genera con **`lxml.etree`** a partir de la plantilla de Moodle (`plantilla_vm.xml`):

```
        arbol = etree.parse("plantilla_vm.xml")
        raiz = arbol.getroot()
        raiz.find("name").text = "agr-rc"
        raiz.find("memory").text = raiz.find("currentMemory").text = str(384 * 1024)  #
KiB
        raiz.find("vcpu").text = "1"
        dev = raiz.find("devices")
        dev.find("disk/source").set("file", "/var/lib/libvirt/images/agr/rc.qcow2")
        dev.remove(dev.find("interface"))                    # se sustituye por una por
red
        for j, itf in enumerate(interfaces_de_rc):
            e = etree.Element("interface", type="bridge")
            etree.SubElement(e, "mac", address=f"52:54:00:aa:03:{j:02x}")
            etree.SubElement(e, "source", bridge=itf["bridge"])
            etree.SubElement(e, "target", dev=f"rc-{itf['nombre']}")   # tap con nombre
legible
            etree.SubElement(e, "model", type="virtio")
            dev.insert(posicion + j, e)
        canal = etree.SubElement(dev, "channel", type="unix")          # agente QEMU
        etree.SubElement(canal, "target", type="virtio", name="org.qemu.guest_agent.0")
        dev.find("graphics").set("keymap", "es")                        # VNC con teclado
español
```

Cambios respecto a la plantilla:

Elemento	Plantilla	Generado
`<name>`	`XXX`	`agr-<máquina>`
`<description>`	—	rol e IPs de la VM
`<memory>`, `<currentMemory>`	1 GB	384 MB (768 MB el servidor)
`<vcpu>`	2	1
`<emulator>`	`/usr/bin/kvm`	igual (o `qemu-system-x86_64` si no existe)
`<disk>/<source>`	`/mnt/tmp/XXX/XXX.qcow2`	disco diferencial de la VM
`<interface>`	una, bridge `XXX`	una por red, con MAC fija y tap `rc-ge0-0-1`
`<channel>`	—	agente QEMU
`<graphics>`	VNC	VNC con `keymap="es"`
`type` del dominio	`kvm`	`kvm` (`qemu` si no hay `/dev/kvm`)

Los XML resultantes están en `doc/development/parte2/xml_generados/` y validan con `virt-xml-validate`. Fragmento de `agr-rc.xml`:

```
<domain type="kvm">
  <name>agr-rc</name>
  <description>AGR lab 3103 - RC (router): ge0-0-5=10.2.0.2/30, ...</description>
  <memory>393216</memory>
  <vcpu>1</vcpu>
  <devices>
    <disk type="file" device="disk">
      <driver name="qemu" type="qcow2"/>
      <source file="/var/lib/libvirt/images/agr/rc.qcow2"/>
      <target dev="vda" bus="virtio"/>
    </disk>
    <interface type="bridge">
      <mac address="52:54:00:aa:03:00"/>
      <source bridge="agr-bc"/>
      <target dev="rc-ge0-0-5"/>
      <model type="virtio"/>
    </interface>
    ... (ge0-0-1 en agr-red1, ge0-0-2 en agr-red2)
    <channel type="unix">
      <target type="virtio" name="org.qemu.guest_agent.0"/>
    </channel>
  </devices>
</domain>
```

Paso 7. Definir y arrancar

El script registra y arranca cada dominio:

```
virsh define /var/lib/libvirt/images/agr/agr-rc.xml
virsh start agr-rc
```

libvirt crea un tap por interfaz (`rc-ge0-0-5`, ...) y lo conecta al bridge indicado. Es el equivalente a enchufar un cable al switch.

Paso 8. Qué ocurre dentro de cada VM al arrancar

1. **`agr-net.service`** se ejecuta antes que la red y que FRR (`Before=network-pre.target frr.service`). El script `agr-net.sh`:
   - Lee `/etc/agr/node.conf` y espera a que existan todas las interfaces.
   - Busca cada interfaz por su MAC en `/sys/class/net/*/address`.
   - La **renombra** (`ip link set dev eth0 name ge0-0-5`), le asigna su IP y la levanta.
   - Activa el reenvío IP (`/proc/sys/net/ipv4/ip_forward = 1`) sólo si el rol es `router`.
   - Instala las rutas con `ip route replace` en PCs y servidor. En los routers se las deja a FRR.
2. **FRR** (sólo routers). `zebra` y `staticd` cargan `frr.conf` e instalan las rutas estáticas en el kernel (aparecen como `proto static`). Después, la unidad complementaria reaplica la configuración con `vtysh -b`.
3. **`ssr-server.service`** (sólo SRV) lanza `node app.js` en el puerto 3000.
4. **`qemu-guest-agent`** se conecta al canal virtio, lo que permite al anfitrión ejecutar órdenes dentro de la VM.

5. Al iniciar sesión, `/etc/profile.d/agr-info.sh` muestra rol, interfaces, rutas y el estado de FRR o del servidor.

Paso 9. Espera y verificación

`crear` espera a que cada VM esté lista:

- el agente QEMU responde;
- `agr-net.service` ha terminado;
- tiene **todas** sus rutas instaladas.

La orden `probar` lanza las pruebas dentro de las VMs mediante el agente; por ejemplo, para el ping de PC1 al servidor ejecuta:

```
        virsh qemu-agent-command agr-pc1 \
          '{"execute":"guest-exec","arguments":{"path":"/bin/sh","arg":["-c","ping -c2
10.100.0.11"],"capture-output":true}}'
        virsh qemu-agent-command agr-pc1 '{"execute":"guest-exec-status","arguments":
{"pid":<pid>}}'
```

Resultado obtenido: **50/50 pruebas correctas** (`doc/development/parte2/pruebas.txt`).